MODELACIÓN Y DISEÑO DE SISTEMAS • MYD 2026

Patrones Creacionales: Singleton y Multiton

Análisis canónico de patrones GoF y aplicación práctica en una plataforma de E-Commerce & Delivery Multi-País.

Grupo N° 17

Integrantes: Alvarez Garcia, Flores, Ruiz, Valor

Intención y Familia GoF

0-2 MIN • BLOQUE 1

Patrón Singleton

> *"Garantizar que una clase tenga una única instancia y proporcionar un punto de acceso global a ella."*
> ***— Gamma, Helm, Johnson, Vlissides (GoF, 1994)***

- **Familia:** Creacional.
- **Propósito:** Control estricto de la instanciación de un recurso crítico global.
- **Mecanismo:** Encapsula la creación dentro de la propia clase y restringe la instanciación externa.

Patrón Multiton

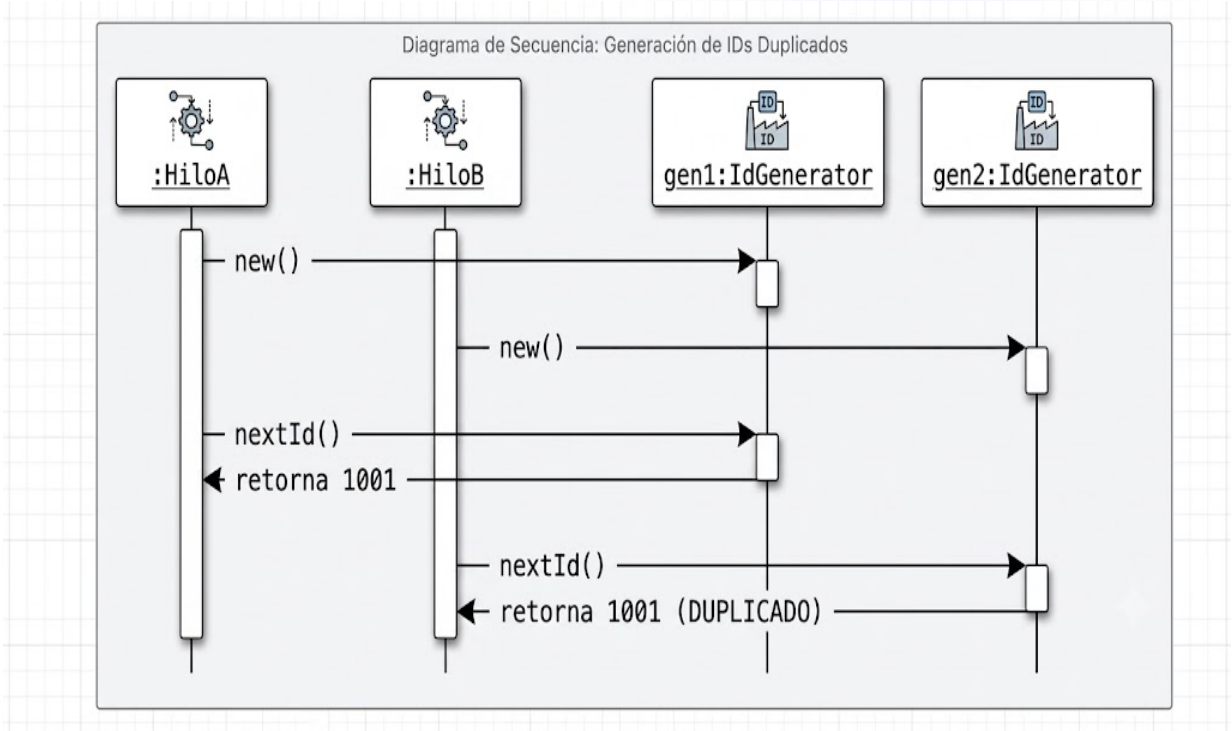
> *"Generalizar el Singleton para garantizar una única instancia por clave, controlando su creación desde un registro central."*
> ***— Refactoring.Guru / Java Design Patterns***

- **Familia:** Creacional (Evolución/Variante del Singleton).
- **Propósito:** Administrar un mapa dictado por claves donde cada clave posee exactamente una instancia única.
- **Mecanismo:** Registro centralizado (`Map `) accesible mediante clave.

Problema sin Singleton: Generación de IDs

2-6 MIN • BLOQUE 2

- **Escenario Multi-País:** Cada servicio de checkout (Argentina, Brasil, México) mantiene su propio contador de pedidos de forma independiente.
- **Colisión Gravísima de Identificadores:** Al arrancar cada contador en 0/1, distintas filiales generan pedidos con el mismo ID (`Pedido #1 AR` y `Pedido #1 BR`).
- **Imposibilidad de Integración:** Al centralizar en el Data Warehouse para facturación o tracking, se producen duplicados destructivos.
- **Escalabilidad Insostenible:** Con N países, la colisión pasa de ser un riesgo a ser una certeza matemática.



Problema sin Multiton: Config Regional

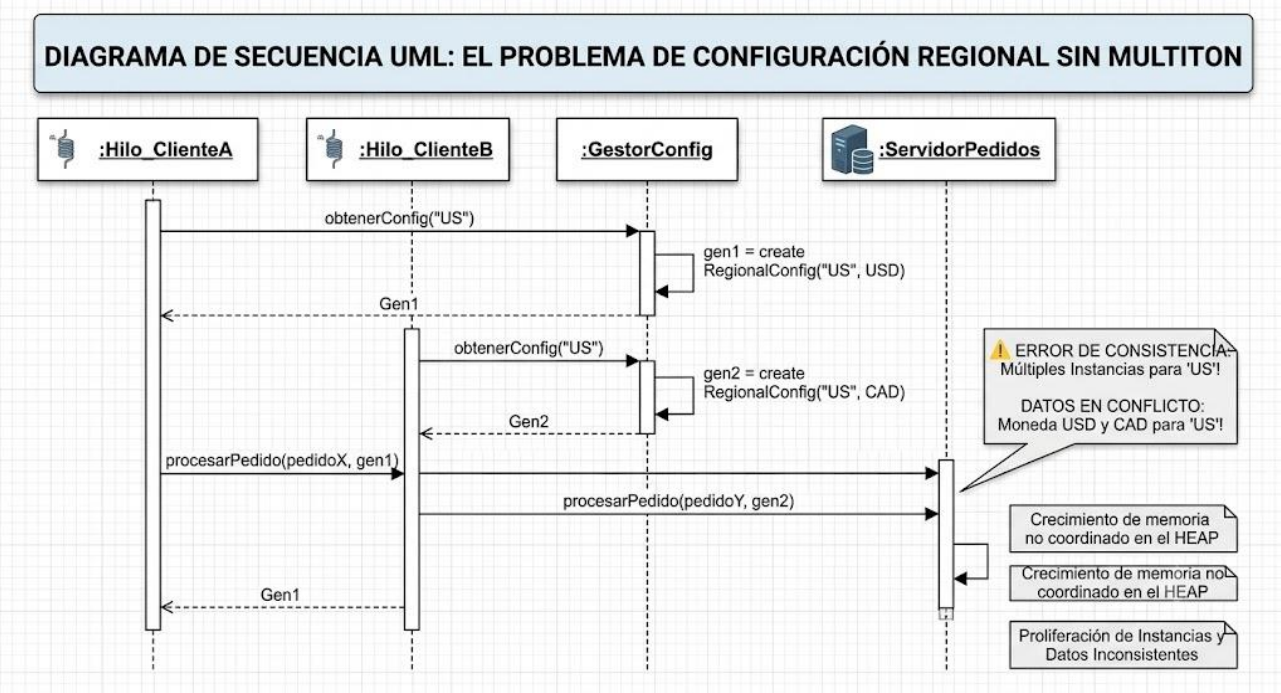
2-6 MIN • BLOQUE 2

Escenario: Los módulos de Checkout, Catálogo y Facturación requieren conocer datos del país (IVA, Moneda, Medios de Pago).

Duplicación de Instancias: Para un mismo país ("AR"), CheckoutModule y CatalogModule mantienen copias aisladas de RegionConfig.

Inconsistencia en Caliente: Si la tasa de IVA cambia en tiempo de ejecución, un módulo puede seguir facturando con el valor desactualizado.

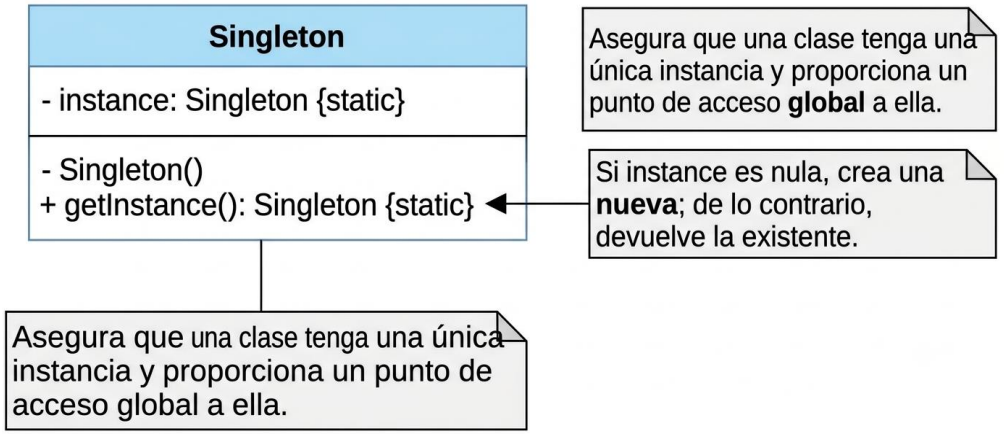
Impacto Multiplicativo: Con N módulos y M países se generan N × M instancias redundantes y consultas excesivas a DB.



Solución Canónica: Patrón Singleton

6-10 MIN • BLOQUE 3

Diagrama de Clases UML del Patrón Singleton (Solución Canónica)




```
class Singleton {
    private static Singleton instancia;

    private Singleton() {
        // constructor privado: nadie fuera de la clase puede instanciar con "new"
    }

    public static Singleton getInstance() {
        if (instancia == null) {
            instancia = new Singleton();
        }
        return instancia;
    }

    public void operacion() {
        // lógica de negocio
    }
}

// Uso:
Singleton s1 = Singleton.getInstance();
Singleton s2 = Singleton.getInstance();
// s1 == s2  ->  true, es la misma instancia
```

1. Constructor Privado

Bloquea categóricamente cualquier intento de instanciación directa desde fuera de la clase mediante la palabra reservada new.

2. Acceso Global

El método estático getInstance() evalúa si la instancia existe; si es nula la crea (Lazy Loading) y devuelve siempre la misma referencia.

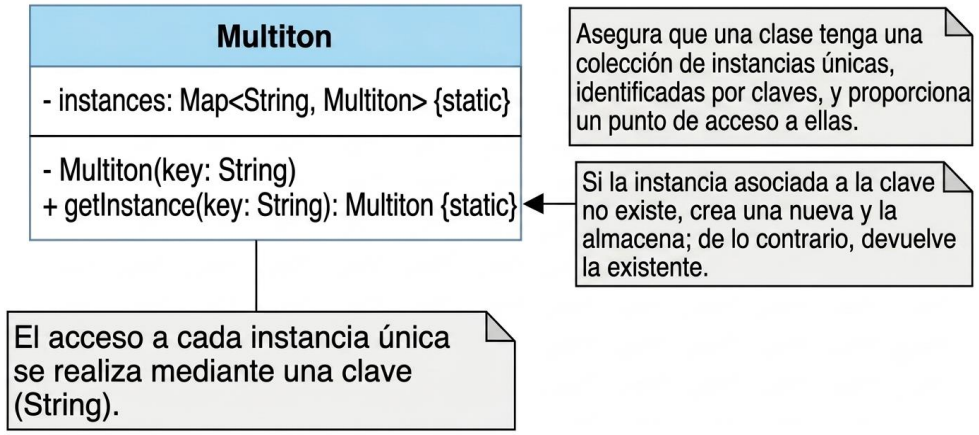
3. Dependencia

Regla unidireccional: **Cliente → Singleton**. El Singleton gestiona su propia existencia y desconoce totalmente a sus clientes.

Solución Canónica: Patrón Multiton

6-10 MIN • BLOQUE 3

Diagrama de Clases UML del Patrón Multiton (Solución Canónica)




```
class Multiton {
    private static Map<String, Multiton> instancias = new HashMap<>();

    private Multiton() {}

    public static synchronized Multiton getInstance(String clave) {
        if (!instancias.containsKey(clave)) {
            instancias.put(clave, new Multiton());
        }
        return instancias.get(clave);
    }

    public void operacion() {
        // lógica de negocio
    }
}

// Uso:
Multiton a1 = Multiton.getInstance("A");
Multiton a2 = Multiton.getInstance("A");
Multiton b1 = Multiton.getInstance("B");
// a1 == a2  ->  true  (misma clave, misma instancia)
// a1 == b1  ->  false (distinta clave, distinta instancia)
```

1. Registro Central

Mantiene un diccionario interno estático (Map) para almacenar y gestionar el ciclo de vida de cada objeto por clave.

2. Acceso Parametrizado

El cliente solicita una instancia indicando la clave: getInstance(clave). Si la clave existe la retorna; de lo contrario, la crea.

3. Unicidad por Ámbito

Garantiza exactamente **1 instancia por clave**. Combina la unicidad del Singleton con la flexibilidad de un registro administrado.

Caso Dominio: E-commerce / Delivery Multi-Pais (Singleton)

10-13 MIN • BLOQUE 4

```java
class OrderIdGenerator {
    private static OrderIdGenerator instancia;
    private long contadorGlobal = 0;

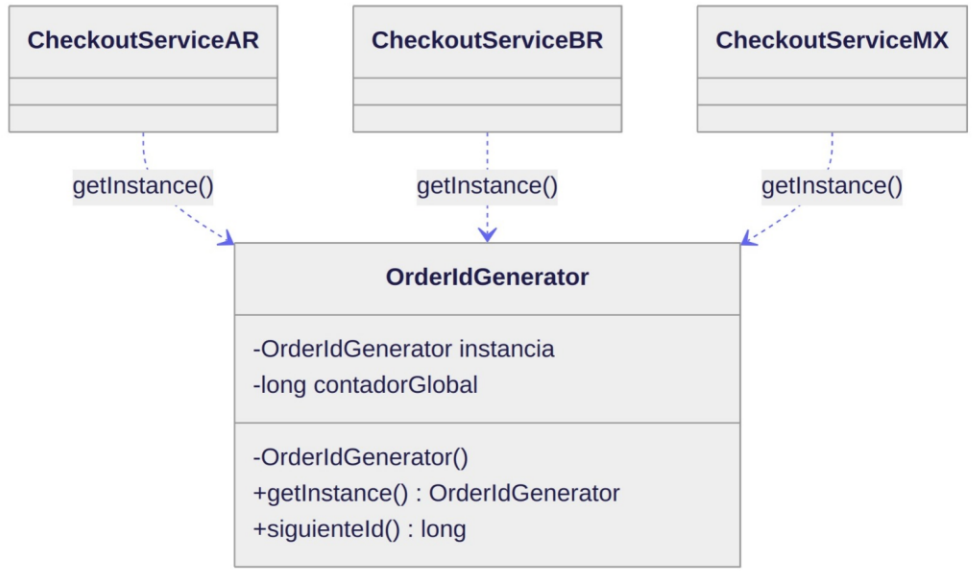
    private OrderIdGenerator() {}

    public static synchronized OrderIdGenerator getInstance() {
        if (instancia == null) {
            instancia = new OrderIdGenerator();
        }
        return instancia;
    }

    public synchronized long siguienteId() {
        contadorGlobal++;
        return contadorGlobal;
    }
}

class CheckoutServiceAR {
    public Pedido crearPedido(Carrito carrito) {
        long id = OrderIdGenerator.getInstance().siguienteId();
        return new Pedido(id, carrito, "AR");
    }
}

class CheckoutServiceBR {
    public Pedido crearPedido(Carrito carrito) {
        long id = OrderIdGenerator.getInstance().siguienteId(); // MISMA instancia que AR
        return new Pedido(id, carrito, "BR");
    }
}
```



Caso Dominio: E-commerce / Delivery Multi-Pais (Multiton)

10-13 MIN • BLOQUE 4

```
class RegionConfigManager {
    private static Map<String, RegionConfigManager> instancias = new HashMap<>();

    private String pais;
    private String moneda;
    private float iva;
    private List<String> metodosPagoHabilitados;

    private RegionConfigManager(String pais) {
        this.pais = pais;
        cargarDesdeDB(pais); // se ejecuta UNA sola vez por pais, sin importar cuántos módulos lo pida
    }

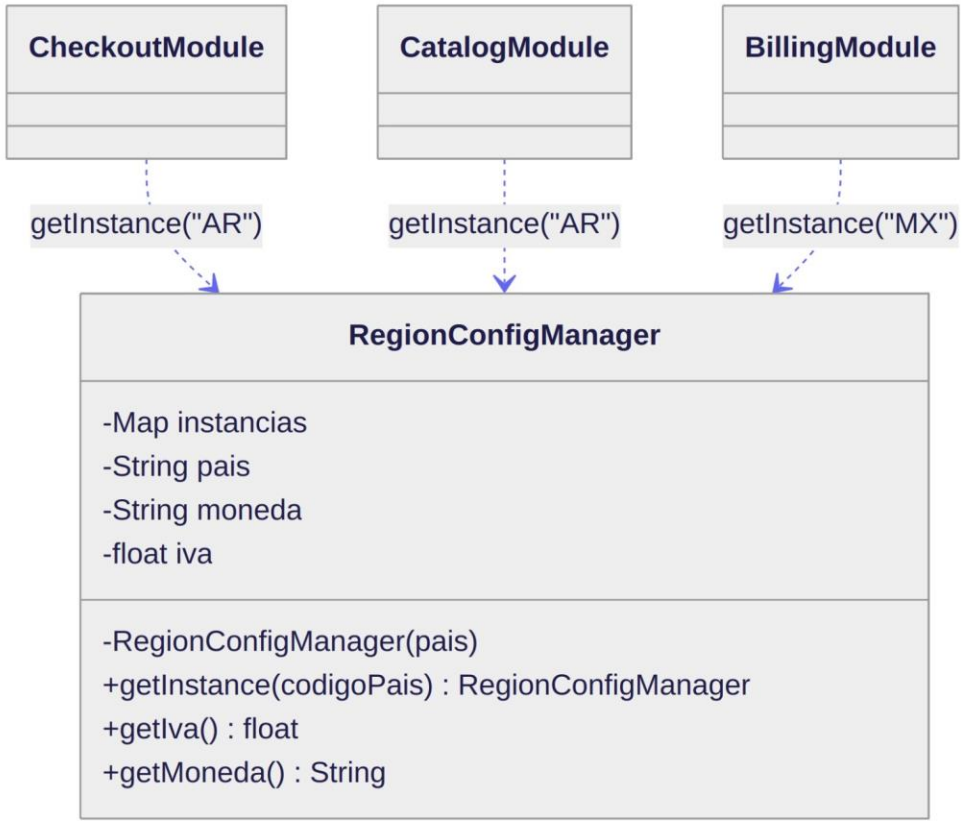
    public static synchronized RegionConfigManager getInstance(String codigoPais) {
        if (!instancias.containsKey(codigoPais)) {
            instancias.put(codigoPais, new RegionConfigManager(codigoPais));
        }
        return instancias.get(codigoPais);
    }

    public float getIva() { return iva; }
    public String getMoneda() { return moneda; }

    // Permite refrescar la config de un pais puntual sin afectar a los demás
    public static synchronized void invalidar(String codigoPais) {
        instancias.remove(codigoPais);
    }
}

class CheckoutModule {
    public void procesarPago(String pais) {
        RegionConfigManager cfg = RegionConfigManager.getInstance(pais);
        // usa cfg.getIva(), cfg.getMoneda()
    }
}

class CatalogModule {
    public void listarProductos(String pais) {
        RegionConfigManager cfg = RegionConfigManager.getInstance(pais);
        // MISMA instancia que ve CheckoutModule para el mismo pais
    }
}
```



Comparativa Directa: Singleton vs Multiton

13-15 MIN • BLOQUE 5

Dimensión de Análisis	Patrón Singleton	Patrón Multiton
Cardinalidad de Instancias	Exactamente 1 por aplicación / JVM	Exactamente 1 por clave / contexto
Punto de Acceso	getInstance() sin parámetros	getInstance(clave) con identificador
Estructura Interna	Referencia estática única (instance)	Mapa asociativo estático (Map)
Caso de Uso en Delivery	Generador global de IDs de Pedido (OrderIdGenerator)	Configuración regional por País (RegionConfigManager)
Criterio de Selección	El concepto es único universalmente en toda la plataforma.	El concepto se repite por cada dominio/inquilino/región.

Cuándo NO usarlos y Anti-Patrones

13-15 MIN • BLOQUE 5

- **Anti-Patrón de Estado Global Mutable:** Oculta las dependencias reales entre componentes. Dificulta el rastreo de fallas al permitir que cualquier módulo modifique el estado interno.
- **Pesadilla para Pruebas Unitarias (Testing):** El estado estático persiste entre ejecuciones de tests unitarios, introduciendo acoplamiento temporal y requiriendo métodos de reseteo no deseados.
- **Sobreingeniería y Acoplamiento:** No deben usarse solo para evitar pasar parámetros en llamadas a métodos. Atan al cliente a clases concretas en lugar de interfaces.
- **Evolución y Alternativa (Inyección de Dependencias):** En arquitecturas modernas se prefiere usar un contenedor IoC (Spring/Guice) con *Scope Singleton*. Esto mantiene la unicidad sin acoplar las clases al método estático getInstance().
- **Trampa del Multiton (Fuga de Memoria):** indexar por una clave limitada (ej. usuariold) haria crecer el Map sin liberar memoria – solo es seguro con Claves finitas (paises). Con una unica clave posible, ya es sobreingenieria; alcanza con un Singleton simple
- **Patrones que compiten (Service Locator, Monostate):** getInstance() es una forma del anti-pattern Service Locator, que oculta dependencias reales. Monostate comparte estado sin restringir la instanciacion - ideal para mockear en tests unitarios.